

The Beginner's Guide: Build and Host a Free Database-Backed Web App

Welcome! If you have ever wanted to build a web application that actually "remembers" your data (like saving tasks on a To-Do list even after you refresh the page), you are in the right place.

We are going to build a **Python Web App** using a framework called **Flask**, connect it to a free **MongoDB** database, and host it live on the internet using **Hugging Face Spaces**.

Don't worry if those words sound intimidating. We will break it down step-by-step.

What You Are Building

You are creating a simple To-Do List. You type a task, hit "Add", and it saves to a real database. You can also click an "X" to delete tasks.

The tools we are using:

- **Python & Flask:** The language and tool used to build the logic of the app.
- **MongoDB Atlas:** A free, cloud-based database that stores your tasks safely.
- **Docker:** A tool that packages your app up so it can run anywhere.
- **Hugging Face Spaces:** A platform that will host your app on the internet for free.

Part 1: Your Project Blueprint

Create a new folder on your computer called mongo-todo-app. Inside that folder, you are going to create four files. Think of this as the blueprint for your app.

Here is what your folder should look like when you are done:

```
/mongo-todo-app
├── app.py      (The brain: Python logic)
├── requirements.txt (The grocery list: Python packages we need)
├── Dockerfile  (The instructions: How to run the app)
├── /templates  (A folder inside your main folder)
└── index.html  (The face: What the user sees)
```



Part 2: The Code

Let's create those four files. Copy and paste the code below into each respective file.

1. app.py

This is the brain of your app. It handles the web traffic, connects to your database, and tells the website what to display.

```
from flask import Flask, render_template, request, redirect, url_for
from pymongo import MongoClient
from bson.objectid import ObjectId
import os

app = Flask(__name__)

# 1. Connect to MongoDB securely using an environment variable
# We do NOT put our password here. We will set MONGO_URI later on Hugging Face.
MONGO_URI = os.getenv("MONGO_URI")
client = MongoClient(MONGO_URI)

# 2. Select the database and the collection (think of a collection like a table)
db = client.todo_database
todos_collection = db.todos

# 3. The Home Page Route
@app.route('/')
def index():
    # Fetch all saved tasks from the database and send them to the webpage
    tasks = list(todos_collection.find())
    return render_template('index.html', tasks=tasks)

# 4. The 'Add Task' Route
@app.route('/add', methods=['POST'])
def add():
    task_content = request.form.get('task')
    if task_content:
        # Save the new task into MongoDB
        todos_collection.insert_one({'task': task_content})
    return redirect(url_for('index'))

# 5. The 'Delete Task' Route
@app.route('/delete/<task_id>')
def delete(task_id):
```

```

# Find the task by its unique ID and delete it
todos_collection.delete_one({'_id': ObjectId(task_id)})
return redirect(url_for('index'))

if __name__ == '__main__':
    # Hugging Face requires apps to run on port 7860
    app.run(host='0.0.0.0', port=7860, debug=False)

```

2. templates/index.html

Inside your templates folder, create index.html. This is the visual interface—the buttons, text boxes, and colors.

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>MongoDB To-Do App</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 40px auto; max-width: 400px; text-align: center; }
        input[type="text"] { padding: 10px; width: 65%; border: 1px solid #ccc; border-radius: 4px; }
        button { padding: 10px 15px; cursor: pointer; background: #198754; color: white; border: none;
border-radius: 4px; }
        button:hover { background: #157347; }
        ul { list-style-type: none; padding: 0; }
        li { background: #f8f9fa; margin: 10px 0; padding: 12px; display: flex; justify-content: space-between;
border-radius: 4px; border: 1px solid #ddd; }
        a { color: red; text-decoration: none; font-weight: bold; }
    </style>
</head>
<body>
    <h2> 🌱 MongoDB To-Do List</h2>

    <form action="/add" method="POST">
        <input type="text" name="task" placeholder="Enter a new task..." required>
        <button type="submit">Add Task</button>
    </form>

    <ul>
        <!-- This is special code that loops through your database tasks -->
        {% for task in tasks %}
            <li>

```

```

        {{ task.task }}
        <a href="/delete/{{ task._id }}" title="Delete Task">X</a>
    </li>
{% endfor %}
</ul>
</body>
</html>

```

3. requirements.txt

This tiny file tells the server exactly which extra Python tools to install before running your code.

```

Flask==3.0.0
pymongo==4.6.1

```

4. Dockerfile

This is a set of instructions. It tells Hugging Face: "Hey, grab Python, install my requirements, copy my files, and start my app."

```

FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 7860
CMD ["python", "app.py"]

```

Part 3: Deploying to the Internet

You have the code. Now, let's put it online!

Step A: Get Your Database Ready

Your app needs a place to save data.

1. Go to [MongoDB Atlas](#) and create a free account.
2. Build a free "Cluster" (a server for your database).
3. Under the **Network Access** tab on the left sidebar, click "Add IP Address" and allow connections from anywhere (0.0.0.0/0). This is necessary because Hugging Face servers change IPs occasionally.
4. Under the **Database Access** tab, create a new Database User. Write down the username and password you create here.

5. Go back to your Cluster, click **Connect**, choose **Drivers**, and copy the connection string.

Important: Your connection string will look something like this:

mongodb+srv://<username>:<password>@cluster0.mongodb.net/?retryWrites=true&w=majority. Make sure you replace <username> and <password> with the exact ones you just created!

Step B: Create Your Hosting Space

Now we need a place to put your code.

1. Create a free account at [Hugging Face](#).
2. Click your profile picture and select **New Space**.
3. Give your space a name (like my-todo-app).
4. Under "Select the Space SDK", choose **Docker** and pick the **Blank** template.
5. Ensure the hardware is set to the free **CPU basic** tier, then click **Create Space**.

Step C: The Secret Password (Crucial!)

We never put passwords directly in our code. We use "Secrets" instead.

1. In your new Hugging Face Space, click the **Settings** tab.
2. Scroll down to the **Variables and secrets** section.
3. Click the **New secret** button.
4. For the Name, type exactly: MONGO_URI
5. For the Value, paste your customized MongoDB connection string from Step A.
6. Click **Save**.

Step D: Upload and Launch

1. Click on the **Files** tab in your Hugging Face Space.
2. Click **Add file** -> **Upload files**.
3. Upload all four items from your folder: app.py, requirements.txt, Dockerfile, and the templates folder.
4. Click **Commit changes** at the bottom.

Congratulations!

Hugging Face will automatically read your Dockerfile, install your tools, connect to your database using the secret key, and launch your app. In a few moments, you will see your live, database-backed web app ready to use!